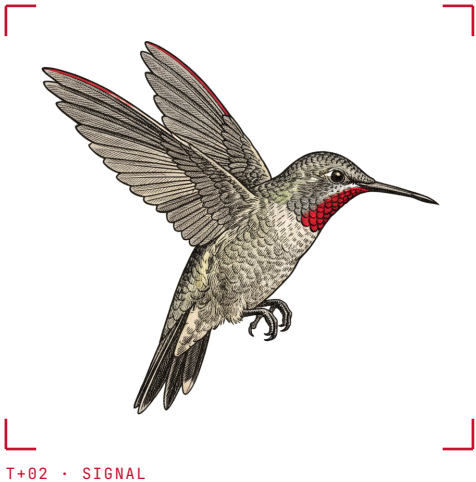


[WORKSHOP] [BUILD-ALONG] [HANDOUT]

GUIDED CODING



PROJECT BRIEF // WEATHER DASHBOARD

Everything you need for the workshop: how to prepare your laptop, what we build, and step-by-step guides for building it three times: in a web chat, in VS Code, and with agents in the terminal.

DATE	TIME	VENUE	SIGN-UP
TUE 29 SEP	17:00	AUDITORIUM	ON THE INTRANET

HOW TO USE THIS HANDOUT

01 BEFORE THE DAY

Work through Section 00, Get ready. About 30–45 minutes, once.

02 ON THE DAY

Follow Sections 01 to 05 in order. Each step says what you should see.

03 KEEP OPEN

Appendix A has prompts to copy. Appendix B has commands and fixes.

CONTENTS

00	Get ready	02	05	Level 3 – CLI agents	10
01	The project	05	06	Working as a team	12
02	The mindset	07	07	Beyond the workshop	12
03	Level 1 – Web chat	08	A	Prompt cheat-sheet	13
04	Level 2 – Editor + memory	09	B	Command card + fixes	14

SECTION 00

GET READY

BEFORE THE WORKSHOP • 30-45 MIN • ONCE

Do this at home or at your desk before the day, so the workshop time goes into building. Follow the steps in order and copy commands exactly. If a step fails, copy the full error message and bring it along; we will fix it together at the start.

0.1 PICK YOUR SETUP

YOU HAVE	DO THIS	THEN
Windows 10 / 11 laptop	Install WSL2 with Ubuntu (0.2). This gives you a real Linux terminal inside Windows.	0.3 onwards, working inside Ubuntu
Linux PC (Ubuntu / Debian)	Nothing extra: you already have a Linux terminal.	Skip to 0.3
Mac	Use the Terminal app. Install tools with Homebrew instead of apt.	0.3 onwards, same ideas

[NOTE] Company laptop? Installing WSL needs administrator rights. Ask IT a few days before the workshop, not on the day.

0.2 WINDOWS ONLY: INSTALL WSL2 + UBUNTU

- 01 Open PowerShell as administrator: press Start, type **PowerShell**, right-click it and choose **Run as administrator**. Then run:

```
POWERSHELL (ADMIN)
wsl --install
```

- 02 Restart the computer when asked. Ubuntu opens by itself and asks for a new **username** and **password**. Pick something simple you will remember: you need the password whenever a command starts with **sudo**.

[NOTE] Nothing appears on screen while you type the password. That is normal; type it and press Enter.

- 03 Check that you are on WSL version 2:

```
POWERSHELL
wsl -l -v
```

[CHECK] The VERSION column next to Ubuntu shows **2**.

[IF IT BREAKS] **“Virtualization is not enabled”**: turn on virtualization in the BIOS/UEFI settings, or ask IT. **“WSL is already installed”** but no Ubuntu: run `wsl --install -d Ubuntu`.

[NOTE]

From now on, **terminal** means the **Ubuntu** app (Start → Ubuntu), not PowerShell. Keep your projects inside Ubuntu's home folder `~`, not under `/mnt/c/`: it is much faster and avoids permission problems.

0.3 INSTALL VS CODE AND OPEN IT ON LINUX

- 01 Install VS Code from **code.visualstudio.com** (on Windows, install the normal Windows version: it connects into Ubuntu for you).
- 02 In VS Code open Extensions (Ctrl+Shift+X) and install **WSL** by Microsoft (Windows only) and **Python** by Microsoft.
- 03 Create the workshop folder and open it in VS Code from the terminal:

```
UBUNTU TERMINAL
mkdir -p ~/guided-coding
cd ~/guided-coding
code .
```

[CHECK]

VS Code opens. On Windows the bottom-left corner shows **WSL: Ubuntu**: that is your VS Code session running inside Linux. Open its built-in terminal with Ctrl+`; it is the same Ubuntu terminal.

0.4 INSTALL THE TOOLCHAIN

Run these in the Ubuntu (or Linux) terminal. The first one installs Git, Python and the Python virtual-environment tool:

```
UBUNTU TERMINAL
sudo apt update
sudo apt install -y git curl python3 python3-venv python3-pip
```

Node.js is only needed for the Level 3 agent tools. Install it with **nvm**, which avoids permission problems. Close and reopen the terminal after the first line:

```
UBUNTU TERMINAL
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.7/install.sh | bash
# close and reopen the terminal, then:
nvm install --lts
```

Tell Git who you are (used when you save work with commits):

```
UBUNTU TERMINAL
git config --global user.name "Your Name"
git config --global user.email "you@company.com"
```

0.5 ACCOUNTS YOU WILL NEED

FOR

ACCOUNT

TIP

Level 1 Any AI chat: ChatGPT, Claude.ai or Copilot Chat.

A free account is enough.

FOR	ACCOUNT	TIP
Level 2	A GitHub account signed in to Copilot in VS Code (the Free plan works), and an OpenRouter account with an API key (openrouter.ai → Keys).	Set a credit limit on the key. Models whose name ends in :free cost nothing.
Level 3	One command-line agent you can sign in to: GitHub Copilot CLI (Copilot plan), Claude Code (Claude Pro, Max or Team) or Codex CLI (ChatGPT plan).	No plan? Pair up with someone who has one: that works well.

[NOTE] Treat API keys like passwords: never paste them into a chat, into code, or into Git. Put them only in the tool's settings.

0.6 FINAL CHECK

Run each command. If every line matches, you are ready.

COMMAND	YOU SHOULD SEE
<code>python3 --version</code>	Python 3.10 or newer
<code>node --version</code>	v22 or newer
<code>git --version</code>	git version 2.something
<code>code --version</code>	a version number (VS Code is reachable from the terminal)

Last, a 1-minute test that Python can install packages:

```
UBUNTU TERMINAL
cd ~/guided-coding
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn
```

[CHECK]	The last line ends with Successfully installed ... fastapi ... and your prompt starts with (.venv) . Type deactivate to leave. Done!
-----------	--

SECTION 01

THE PROJECT

WHAT WE BUILD · THE SAME APP, THREE TIMES

A tiny full-stack **Weather Dashboard**: type a city, see its current weather. A Python backend fetches the data, a web page shows it. You build it three times, each time with a more capable AI workflow, each in its own folder.

1.1 WHAT IT MUST DO

- 01 **Backend endpoint** GET /weather?city=Paris returns JSON with the city name, temperature in °C, a short condition (e.g. "Light rain") and wind speed.
- 02 It uses **Open-Meteo** (open-meteo.com): the free geocoding API turns the city name into coordinates, the forecast API gives the weather. No API key, no signup.
- 03 **Frontend page** with a text box, a **Get weather** button and a result card.
- 04 **Friendly errors**: an unknown city or a stopped backend shows a message, never a crash.
- 05 **CORS enabled** in the backend for http://localhost:5500. Without it the browser blocks the page from calling the backend. Just include this line in your spec; the AI knows how to add it.

1.2 TECH STACK

PART	CHOICE	WHY
Backend	Python 3 + FastAPI + uvicorn (+ httpx for web requests)	A few lines of code, and a free test page at /docs
Frontend	One index.html using React from a CDN	No build tools. (Know Vite already? Fine, use it.)
Data	Open-Meteo geocoding + forecast	Free, no key, no signup

1.3 FOLDER LAYOUT

```
FOLDER TREE
~/guided-coding/
├── level1/
│   ├── backend/
│   │   ├── main.py           # FastAPI app with /weather
│   │   └── requirements.txt  # fastapi, uvicorn, httpx
│   ├── frontend/
│   │   └── index.html        # the page (React via CDN)
│   └── README.md             # how to run it
├── level2/                   # same layout + PLAN.md, MEMORY.md
└── level3/                   # same layout + AGENTS.md, TASKS.md
```

1.4 HOW TO RUN IT (SAME AT EVERY LEVEL)

You need **two terminals**: one for the backend, one for the frontend. In VS Code, click the **+** in the terminal panel to open a second one. Replace `level1` with the level you are on.

01 Terminal 1: backend. The first two lines are only needed once per level.

```
TERMINAL 1
cd ~/guided-coding/level1/backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload
```

[CHECK] Open <http://localhost:8000/weather?city=Paris> in your browser: you see JSON.
<http://localhost:8000/docs> gives you a clickable test page.

02 Terminal 2: frontend.

```
TERMINAL 2
cd ~/guided-coding/level1/frontend
python3 -m http.server 5500
```

[CHECK] Open <http://localhost:5500>, search for a city, see the card.

[NOTE] Stop a server with `Ctrl+C`. On Windows, your normal Windows browser can open these localhost addresses; WSL forwards them automatically.

1.5 DEFINITION OF DONE

- [] The backend returns correct JSON for at least three different cities.
- [] The page shows the weather card for a city you type.
- [] An unknown city shows a friendly message instead of an error.
- [] README.md explains how to start the backend and the frontend.

SECTION 02

THE MINDSET

APPLIES AT EVERY LEVEL

The tools change at each level; these habits don't. They are adapted from Darren Coxon's "Four Ps of Vibe Coding". The memory files you meet later (PLAN.md, MEMORY.md, AGENTS.md) are these habits, written down.

2.1 THE FOUR PS

P	WHAT IT MEANS	IN PRACTICE
Plan	Get the plan and folder structure before any code.	"Don't write code yet. First give me the plan and the file tree."
Project-manage	You are the project manager; the AI forgets what it built.	Ask for one file at a time, full file contents, then test before moving on.
Patience	Things will break, often for confusing reasons. That is normal.	Most people quit in the first hour. Don't be most people.
Persistence	Debugging is a skill you practise, not bad luck.	Use the debugging loop below until it is green.

2.2 THE DEBUGGING LOOP

- 01 Run the code.
- 02 Copy the exact error: all of the red text in the terminal, or in VS Code hover the red squiggly underline and copy the message.
- 03 Paste it back to the AI and ask for a fix (prompt A4 in the appendix).
- 04 Re-run. Repeat until the error is gone. Don't guess, and don't jump to "rewrite everything".

2.3 TOKENS AND CONTEXT, IN 30 SECONDS

- A **token** is a few characters. Your prompt, pasted files and the reply all use tokens.
- The **context window** is the AI's short-term memory. Go past it and earlier instructions are silently forgotten.
- So: keep prompts small and focused, and keep key facts in a short file you can hand back to the AI. Cheaper and more reliable.

SECTION 03

LEVEL 1 – WEB CHAT

ABOUT 40 MIN • FOLDER level1/ • CHAT + VS CODE + TERMINAL



The AI is only a text generator in a browser tab. You create every file, paste every line and run everything yourself. You will learn to scope a request and to read AI output critically.

3.1 STEPS

- 01 Create the folders and open them in VS Code:

```
UBUNTU TERMINAL
mkdir -p ~/guided-coding/level1/backend ~/guided-coding/level1/frontend
cd ~/guided-coding/level1
code .
```

- 02 Open a new chat. Paste **Section 01** (1.1 to 1.3) together with prompt **A1**.

[CHECK] You get a plan and a file tree, and no code yet.

- 03 Ask for **one file at a time**, in this order: backend/requirements.txt, backend/main.py, frontend/index.html (prompt **A3**). In VS Code right-click the folder → **New File**, paste the full content, save with Ctrl+S.

- 04 Run it exactly as in **1.4**.

[CHECK] The page shows a weather card for your city.

- 05 **Debugging drill.** Break something on purpose: rename a variable in main.py and save. Copy the whole red error from Terminal 1, paste it to the chat with prompt **A4**, apply the fix, re-run.

- 06 **Context-recovery drill.** Put all your files into one text file:

```
UBUNTU TERMINAL
cd ~/guided-coding/level1
tail -n +1 backend/main.py backend/requirements.txt \
  frontend/index.html > project.txt
```

Open a **brand-new** chat, paste prompt **A5** and the contents of project.txt.

[CHECK] The AI describes your project correctly without seeing the old chat.

[IF IT BREAKS] error: externally-managed-environment or ModuleNotFoundError: fastapi: the virtual environment is not active. Run source .venv/bin/activate in the backend folder. Browser says **blocked by CORS policy**: ask the AI to “add CORS middleware allowing http://localhost:5500”.

3.2 TALK ABOUT

- The chat has no memory between conversations: you re-supply the context every time.
- It can't run your code, so every fix is a guess until you test it.
- Very fast to start, but it doesn't scale past a small project.

SECTION 04

LEVEL 2 – EDITOR + MEMORY

ABOUT 45 MIN • FOLDER level2/ • VS CODE AI EXTENSION + OPENROUTER



The AI now lives inside VS Code: it sees your files and proposes edits, which you review and accept. Two small Markdown files give it a memory, and OpenRouter lets you switch models.

4.1 SETUP

- 01 Install **one** AI extension from the Extensions panel: **GitHub Copilot Chat** (sign in with GitHub), or **Cline** / **Continue** if you want to use OpenRouter directly.
- 02 Connect OpenRouter. In Cline or Continue, choose **OpenRouter** as the provider and paste your key. In Copilot Chat, open the model picker → **Manage Models** and add OpenRouter if your version offers it; otherwise use Copilot's built-in models.

[CHECK] You type "hello" in the chat panel and get an answer.

4.2 STEPS

- 01 Create the folders and open them:

```
UBUNTU TERMINAL
mkdir -p ~/guided-coding/level2/backend ~/guided-coding/level2/frontend
cd ~/guided-coding/level2
code .
```

- 02 Create **MEMORY.md**: short, lasting facts, so you never repeat them:

```
MEMORY.md
# MEMORY
- Stack: Python 3 + FastAPI backend, React (CDN) in one frontend/index.html
- Data: Open-Meteo geocoding + forecast, no API key
- Backend on :8000, frontend on :5500, CORS allows :5500
- Always give complete files, never snippets
```

- 03 Paste Section 01 into the chat and ask: *"Read MEMORY.md and this spec. Write PLAN.md as a numbered checklist. No code yet."*

[CHECK] PLAN.md exists, with lines like - [] create backend/main.py.

- 04 Work through PLAN.md **one item at a time**. When the AI proposes an edit, read the green (added) and red (removed) lines before you accept. Test, then tick the box ([x]).
- 05 **Compare two models**. Ask the same question with a free model and a stronger one. Note the speed, the quality and the cost.

- 06 Debugging drill.** Break something. VS Code marks it with a red file name and a red squiggly line. Hover it, copy the message, paste it to the chat.

[NOTE]

Starting a new chat? Begin with: “Read *PLAN.md* and *MEMORY.md*, then continue with the next unchecked item.” This is the Level 2 version of the project.txt trick.

4.3 TALK ABOUT

- Memory files cut repeated explanations, and with them token use.
- Model choice is a trade-off: cost, speed and quality differ by task.
- You still decide what gets accepted: review every edit.

SECTION 05

LEVEL 3 – CLI AGENTS

ABOUT 50 MIN • FOLDER level3/ • AGENT IN THE TERMINAL + GIT



An agent in the terminal reads and writes files and runs commands itself. Your job changes from typing to briefing, approving and reviewing.

5.1 INSTALL ONE AGENT

TOOL	INSTALL	START	SIGN IN
GitHub Copilot CLI	<code>npm install -g @github/copilot</code>	<code>copilot</code>	/login (Copilot plan)
Claude Code	<code>curl -fsSL https://claude.ai/install.sh bash</code>	<code>claude</code>	browser prompt (Pro / Max / Team)
Codex CLI	<code>npm install -g @openai/codex</code>	<code>codex</code>	“Sign in with ChatGPT”

[NOTE]

Never use `sudo` with `npm install -g`: with `nvm` it isn't needed, and it causes permission errors later.

5.2 STEPS

- 01** Create the project as a Git repository and open it:

```
UBUNTU TERMINAL
mkdir -p ~/guided-coding/level3
cd ~/guided-coding/level3
git init
code .
```

- 02** Save **Section 01** of this handout as `SPEC.md` in the folder.

03 Create **AGENTS.md**: the agent's standing instructions:

```

AGENTS.md
# AGENTS.md
## Role
Build the Weather Dashboard described in SPEC.md.
## Rules
- Plan first: write PLAN.md and wait for my OK before coding.
- Backend code only in backend/, frontend code only in frontend/.
- Use a .venv for Python. Never commit .venv/ or any keys.
## Done when
- Every box in SPEC.md section 1.5 is ticked and README.md is written.

```

[NOTE]

Copilot CLI and Codex read AGENTS.md automatically. Claude Code reads CLAUDE.md instead: create one containing the single line @AGENTS.md.

04 Start your agent inside level3 (copilot, claude or codex) and ask: “Read AGENTS.md and SPEC.md, then propose a plan.” Approve the plan, then let it build, run and test. **Read every command before you approve it.**

[CHECK]

The app runs as in 1.4, and git log shows the agent's commits.

05 **Two agents in parallel** (shown by the instructor first). Ask the agent to write TASKS.md split into backend and frontend tasks. Open two terminals, start one agent in each and tell them: “You own backend/ only” and “You own frontend/ only”. They coordinate through TASKS.md.**06** **Optional extras. MCP servers** give an agent new tools (for example GitHub, to open a pull request). **claude-code-router** sends planning to a strong model and routine edits to a cheaper one: npm install -g @musistudio/claude-code-router, then start with ccr code.**5.3 GUARDRAILS**

- Review every change the agent makes before it is committed or merged.
- Limit what it can touch: keep permission prompts on and read them.
- Keys live in environment variables and .gitignore, never in the code.
- More autonomy is not less responsibility: you are now the architect and the reviewer.

SECTION 06

WORKING AS A TEAM

WHEN SEVERAL PEOPLE AND AGENTS SHARE ONE REPO

PRACTICE	WHAT IT MEANS
Tracker file	A TASKS.md listing each task, its owner and status. Humans and agents check it before starting and update it when done.
Isolation	Each person or agent owns one folder or module, and works on their own branch.
Pull requests	Every change, human or agent, goes through a pull request and is reviewed. Nobody pushes straight to main.
CI	Lint, tests and build run automatically on every pull request. A red CI result is feedback to fix before merging.

SECTION 07

BEYOND THE WORKSHOP

THE OTHER AXIS: PRODUCT MATURITY

Today you climbed the **tooling** ladder. When a project becomes real, a second, independent ladder matters: how mature the product is (after FreshVanRoot’s “Levels of VibeCoding”).

STAGE	TYPICAL TOOLS	WHAT MATTERS MOST
Prototype	Lovable, Bolt, or any level from today	Speed. Throwaway is fine.
MVP	Replit, Cursor, Level 2 / 3 workflows	A foundation you won’t have to rewrite.
Production	Level 3 + an experienced developer reviewing	Security review, a deployment plan, no blind pushes.

Pick the tooling that fits how careful you need to be, not the one that feels most advanced.

SECTION A

PROMPT CHEAT-SHEET

COPY, PASTE, FILL IN THE [BRACKETS]

A1 PLAN FIRST, NO CODE

PROMPT

We will build [describe the project]. Do NOT write any code yet.
First give me the plan (the stages we will go through)
and the folder structure as a tree.

A2 EXPLAIN LIKE I'M NEW

PROMPT

Assume I have no prior knowledge of [tool / step]. Explain it step by step, like an instruction manual for a complete beginner.

A3 ONE COMPLETE FILE AT A TIME

PROMPT

Now give me the complete content of [path/file].
Only this one file, the full file, no snippets.
Then stop so I can add and test it.

A4 FIX AN ERROR

PROMPT

I ran [command] and got this exact error:
[paste the full error]
Please fix it, give me the complete corrected file,
and explain in one sentence what was wrong.

A5 RECOVER CONTEXT IN A NEW CHAT

PROMPT

Here is the current state of my project (all files below).
Read it, summarise what it does and how it is structured,
and wait for my next instruction.
[paste project.txt]

A6 RESUME WITH MEMORY FILES (LEVEL 2 / 3)

PROMPT

Read PLAN.md and MEMORY.md (or AGENTS.md).
Tell me the next unchecked item, then do only that item.

SECTION B

COMMAND CARD + FIXES

KEEP THIS OPEN WHILE YOU WORK

B.1 COMMANDS YOU WILL USE

TO...	TYPE
Go to a level's folder	<code>cd ~/guided-coding/level1</code>
Open the folder in VS Code	<code>code .</code>
Create the Python environment (once)	<code>python3 -m venv .venv</code>
Switch it on (every new terminal)	<code>source .venv/bin/activate</code>
Install the backend's packages	<code>pip install -r requirements.txt</code>
Start the backend	<code>uvicorn main:app --reload</code>
Start the frontend	<code>python3 -m http.server 5500</code>
Stop a running server	<code>Ctrl+C</code>
See what changed / save a snapshot	<code>git status · git add . then git commit -m "msg"</code>

B.2 IF IT BREAKS

YOU SEE	DO THIS
error: externally-managed-environment	You are installing outside a virtual environment. Run <code>source .venv/bin/activate</code> first (create it once with <code>python3 -m venv .venv</code>).
ModuleNotFoundError: No module named 'fastapi'	Same cause: activate <code>.venv</code> . Your prompt should start with <code>(.venv)</code> .
[Errno 98] Address already in use	A server is already running. Find its terminal and press <code>Ctrl+C</code> , or add <code>--port 8001</code> .
Browser console: blocked by CORS policy	Ask the AI: "add CORS middleware to the FastAPI app allowing <code>http://localhost:5500</code> ".
Page is blank or does nothing	Press <code>F12</code> → Console. Copy the red error into the AI (prompt A4).
code: command not found (WSL)	Close and reopen Ubuntu. Check the WSL extension is installed in VS Code.
nvm: command not found	Close and reopen the terminal (or run <code>source ~/.bashrc</code>).
npm: EACCES permission denied	Node was not installed with nvm. Redo the nvm step in 0.4; never use <code>sudo npm</code> .